Continuous Compliance Monitoring of Water Treatment Plants Using a Fog-to-Cloud Architecture

Rakesh Kunchala
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
Student ID: X25176862
x25176862@student.ncirl.ie

***Abstract*—A water treatment plant must hold several quality parameters inside regulated bands continuously, yet an operator who learns of a breach only from a periodic grab sample may act hours after the water has already left the plant. This paper presents a working fog-to-cloud pipeline that watches two treatment plants without interruption. Each plant carries five instruments, turbidity, pH, free chlorine, flow rate, and line pressure, that are sampled locally and dispatched to a fog gateway at the plant. At the gateway each stream is grouped into a fixed window, condensed to summary statistics, and checked against compliance rules that encode recognised drinking-water limits, so a breach shows up in the cycle it occurs rather than after a laboratory turnaround. Each window's compact aggregate travels over a managed message queue to an event-driven ingestion function that records it in a key-value store, while a separate, independently deployed function answers a dashboard interface behind a managed gateway and the operator page loads as static content from object storage. The system is built in Node.js and provisioned to a real Amazon Web Services account by a single infrastructure-as-code apply that created twenty-four resources. Readings from the live endpoint show the pipeline healthy end to end within roughly a minute of start-up and the stored record count climbing poll after poll, with both plants rendering live, compliance-annotated readings in a browser. A defect that surfaced only in the deployed function, an argument-shape collision on the serverless runtime, is documented together with its fix.**

***Index Terms*—fog computing, edge computing, water quality monitoring, regulatory compliance, serverless, message queue, Internet of Things.**

I. Introduction

Treated drinking water is a regulated product. A plant is expected to keep turbidity low, keep a disinfectant residual present throughout the distribution network, hold pH within a narrow band, and maintain line pressure, and it is accountable for doing so continuously rather than on average. The World Health Organization's drinking-water guidelines set out the parameters that matter and the reasoning behind their limits: turbidity must be low both because it is an aesthetic and operational concern and because particulate matter shields microorganisms from disinfection; a free chlorine residual must be maintained through the network to guard against recontamination; and pH must be controlled because disinfection efficiency and pipe corrosion both depend on it [1]. The common thread is that these are not one-off acceptance tests but conditions that must hold at all times.

Traditional monitoring does not match that requirement well. A grab sample sent to a laboratory characterises the water at one instant and reports back after a delay, so a transient excursion, a chlorine dip after a dosing fault, a turbidity spike after a filter breakthrough, can pass entirely between samples, or be discovered only once the affected water has already been distributed. Continuous instrumentation closes that gap, and a body of work has shown that networked sensors can monitor water quality in real time and at acceptable cost [2], [3]. The open question for a systems designer is not whether to instrument the plant but where the resulting data should be processed and how it should travel to the people who act on it.

Streaming every raw reading to a central cloud is the wrong default. A plant with several fast instruments produces readings faster than any of them is individually useful, the uplink from a remote plant is finite and occasionally unreliable, and a monitoring view that goes dark whenever connectivity dips is of least use during exactly the upsets that stress the plant. Edge and fog computing answer this by shifting work back toward where the data originates [4]. A fog tier placed between the instruments and the cloud lets the windowing, the summarising, and the time-sensitive rule checks run at the plant, and leaves the cloud to do what it does best, hold data durably, serve queries elastically, and present a globally reachable interface [5], [6]. The cloud services drawn on here fit the familiar on-demand, elastic, metered model of cloud computing [7].

This paper reports the design, implementation, and live deployment of such a pipeline for water treatment monitoring. The deployment watches two plants, each carrying five instruments. The objectives are fourfold. First, to reduce raw readings to compact per-window aggregates at a fog tier at the plant, so that only summaries cross the link to the cloud. Second, to evaluate compliance rules that encode recognised drinking-water limits at the edge, so that a breach is flagged in the cycle it is observed. Third, to build a cloud back end that ingests aggregates reliably, stores them durably, and serves them to an operator dashboard without the write path and the read path contending. Fourth, to deploy the whole system to a real cloud account and verify it end to end against the live endpoint. The sections that follow cover the architecture and

the decisions behind it, the implementation with its automated tests, the deployment together with a defect that showed itself only once deployed, an evaluation taken from the running system, and a closing reflection.

II. System Architecture and Design

The architecture follows a single ingestion path with a branch for the operator interface. The instruments emit readings, the plant-side fog gateway groups them into windows, reduces them, and checks them against limits, a queue moves the resulting summaries into the cloud, one function writes them to storage, and a separate function reads them back for the dashboard. The deployed layout appears in Fig. 1. Each tier is described below, followed by the reasoning behind the main choices.

A. Edge Sensing and Fog Aggregation

Every plant carries five sensor processes, one for each measured quantity, turbidity in nephelometric turbidity units, pH, free chlorine in parts per million, flow rate in litres per second, and line pressure in bar. To produce a realistic signal, a sensor walks its value by a bounded step, nudging each reading a little away from the last and holding it inside a sensible range so the trace wanders instead of jumping; the hydraulic streams take a larger step than the chemical ones, matching how much faster they move between samples. Two timers drive every sensor, one deciding the sampling rate, which defaults to a reading every two seconds, and the other the rate at which the collected readings are pushed to the gateway, which defaults to a dispatch once at least ten seconds have passed since the last. Decoupling those two rates means a lively quantity can be sampled densely yet still travel to the edge in compact batches, which is how field instruments are typically tuned. A push is a short JSON body carrying the sensor type, the plant, the unit, and the handful of timestamped values gathered since the last push, altogether only a few hundred bytes.

Built on Node.js using the runtime's own built-in HTTP server with a small custom route table rather than a web framework, the fog gateway is deliberately small. Incoming batches are appended to a per-window ledger of readings, grouped by sensor-and-plant pair. At a fixed interval it seals the open window and reduces each group's buffered readings to a summary that carries the window boundaries, the count, the minimum, the maximum, the mean, and the last value, and it runs that summary through the compliance rule for its sensor type before letting it move on. The readings of a window are held only until that boundary, when the ledger is cleared for the next window rather than being allowed to grow without bound.

B. Compliance Rules at the Edge

Four rules are evaluated, one per compliance-relevant stream; flow rate is retained for display and trend analysis but carries no rule, because a high or low instantaneous flow is an operational quantity rather than a quality breach. Table I lists the rules alongside the drinking-water rationale each encodes. Three rules watch the window mean, because a sustained average is a more meaningful signal of a genuine excursion than a single noisy sample; the pressure rule instead watches the window minimum, because the worst reading in the window, not its average, is the signal that distribution integrity may have been lost. Evaluating these rules at the edge means a breach is detected in the same cycle as the readings that reveal it, with no round trip to the cloud, which is precisely the property a laboratory grab sample cannot provide.

TABLE I
Edge-Evaluated Compliance Rules

Stream	Rule (per window)	Breach raised
Turbidity	mean above 5 NTU	Turbidity alert
Free chlorine	mean below 0.2 ppm	Under-chlorination
pH	mean below 6.5	Acidic violation
Line pressure	minimum below 2 bar	Low-pressure fault
Flow rate	(informational, no rule)	None

At the close of a window the gateway avoids sending a separate message for each plant and stream. Instead it gathers all the summaries from that cycle and pushes them in one batched call to the queue, breaking the set into smaller groups only where the queue caps how many entries a single call may carry. With two plants yielding as many as ten summaries per cycle, ten separate cloud calls collapse into one, which trims both request cost and the fixed overhead paid per message.

C. Decoupled Cloud Ingestion and Storage

Rather than call the storage function outright, the fog tier posts its aggregates onto a managed message queue. That queue absorbs timing differences between the two sides: messages sit durably until something consumes them, so a consumer that is slow or momentarily down cannot lose a summary or stall the gateway, and a sudden rush of aggregates is levelled to a pace the consumer can keep up with, which is the classic producer-consumer decoupling of message-oriented middleware [8]. That guarantee applies once a message is on the queue, not to the act of enqueuing it: should the gateway's batched call fail, the window is already sealed and its ledger cleared, so rather than retry or hold the data back the gateway records the failure and carries on to the next window. This is an accepted compromise given how little data two plants generate, with a fresh window only seconds away.

The queue is drained by an event-driven ingestion function, handed a batch of messages, that reshapes each aggregate into a stored record and puts it into a managed key-value table. Records are keyed in two parts. The partition key is the sensor type; keeping every reading for one quantity together lets the frequent request, the recent windows for a given stream, resolve in a single lookup. The sort key joins the window-end timestamp to the plant label. Ordering by timestamp keeps a partition in chronological order, and tacking on the plant label matters for a specific reason: two plants sealing the same stream in the same cycle produce records that agree on both

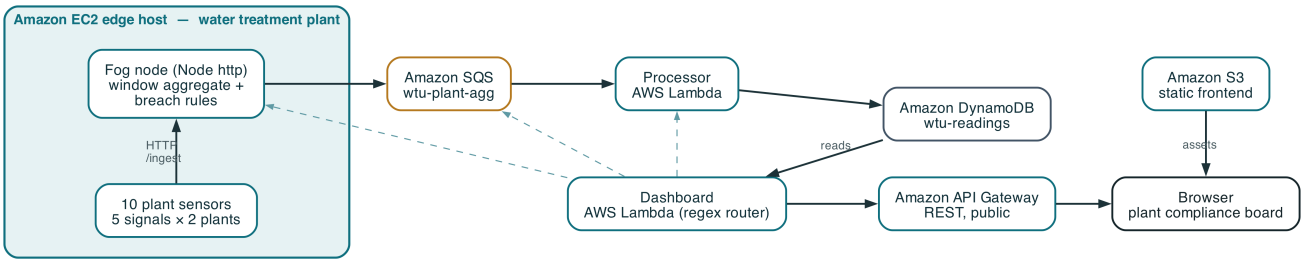

Fig. 1. Deployed fog-to-cloud topology. A single Amazon EC2 edge host at the plant runs the fog gateway and ten sensor containers, five streams at each of two plants. Solid arrows are the ingest path, from the gateway through Amazon SQS and the processor function into the Amazon DynamoDB table, and the serving path, from the dashboard function's read of the stored windows out through Amazon API Gateway to the browser, with static assets from Amazon S3. The dashed edges are the three further status sources the dashboard polls for its health strip: the gateway's reachability, the queue's depth, and the processor function's deployment state.

partition key and window-end time, so without the plant label the later write would clobber the earlier one. Joining the label makes the keys distinct and lets the two plants share a partition safely.

D. Serverless Dashboard Tier and Frontend

A second cloud function, deployed on its own rather than alongside the ingestion function, serves the operator view, so that reads and writes scale and fail without touching one another. It exposes a compact read-only interface: a plants endpoint giving the newest reading for every stream at both plants, a per-plant detail endpoint, a readings endpoint returning a recent series for one stream at one plant to feed the trend charts, a health endpoint, a back-end statistics endpoint, and a thresholds endpoint that forwards the gateway's published rules. Health comes back as four separate flags, whether the gateway responds, whether the queue is reachable, whether the ingestion function is active, and whether the pipeline is current in the sense that the newest stored window is recent, alongside the age of that newest window. Building this response draws on four places at once, the table for readings and counts, the queue for its depth, the ingestion function for its state, and the gateway for health and thresholds, which are the dashed links in Fig. 1. Where the ingestion function does one repetitive job, the dashboard function fans out across several handlers, each a small query against one of those places, chosen by a terse match on method and path.

Access to the dashboard function is through a managed interface gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, one stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content delivery network. On the page a plant-readings table shows each stream's current value with a meter set against its plausible span, a compliance badge per plant, and a set of window-average trend charts placing the two plants side by side, while a header strip shows the four pipeline flags. Every two and a half seconds the script refetches the plants view, health, and back-end statistics and redraws from each reply, keeping the display live without a reload. The page finds the interface origin from one placeholder written into it, replaced with the real gateway origin when the assets are uploaded and read a single time by the script; run locally, where a single process serves both the page and the interface, the placeholder is left blank and requests fall back to the same origin.

E. Design Rationale: Alternatives Considered

None of the choices below was made by default; each has a reason.

Queue instead of a direct call. The gateway could call the storage function straight away and synchronously, which would drop the queue and shave one hop. That option was declined. A synchronous call ties the edge to how available and how fast the cloud function is: throttle it, leave it cold, or take it down briefly, and the gateway stalls or sheds data while the pressure works its way back to the instruments. The queue severs that tie, soaks up spikes, retries on the consumer's behalf, and lets the gateway treat a summary as delivered the moment it lands on the queue [8]. On a window cadence the small added delay does not matter, while the durability and separation are precisely what a shaky plant link calls for.

Key-value table instead of a relational database. A relational database would bring joins and open-ended queries, neither of which this workload uses. Its access is narrow and fixed ahead of time: append aggregates in time order, then fetch the latest windows for a stream. A key-value store made for exactly that mix of heavy writes and key-based reads holds its performance as it grows and avoids the operational cost of running a relational engine, the balance the original Dynamo work set out and the managed offering carries forward [9], [10]. Under the chosen keys, the dashboard's one query becomes a single partition read.

Event-driven functions instead of a standing server. Both the ingestion and the interface tier run as event-driven functions, a good fit for work that arrives in bursts and, for the interface, only now and then: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand moves, with nothing reserved up front. The recognised downside is the cold start after a function has sat idle [11]; its impact here is small, since a steady flow of queue messages keeps the ingestion function warm and the dashboard's frequent polling does the same for the interface.

TABLE II
AUTOMATED TEST COVERAGE BY MODULE

Module	Tests
Sensors	11
Fog gateway	51
Ingestion function	10
Dashboard interface	43
Total	**115**

F. Security Posture

Only one inbound port is open on the edge host, and it carries just the gateway's health and threshold endpoints; the host has no login key pair and is managed entirely over the provider's systems-management channel, which takes away the usual remote-access foothold. The two cloud functions share the account's laboratory role because the account is not allowed to mint new ones; in production the plan is to divide that into least-privilege roles, letting the ingestion function only pull and delete queue messages and write records, and the dashboard function only read. Leaving the dashboard interface open is deliberate, since it publishes nothing but aggregated, non-personal plant telemetry; and because the page's origin differs from the interface's, every reply, error replies included, carries an explicit cross-origin header so the browser will accept a response served across origins [12].

III. IMPLEMENTATION

Node.js runs the whole stack, with no build step and no TypeScript. The fog gateway is served by the runtime's own built-in HTTP server with a small custom route table rather than a web framework, and the cloud clients for the queue, the table, and the functions come from the provider's official software development kit for JavaScript. For the dashboard's trend charts, the charting library is checked into the project and served from there instead of a content delivery network, leaving the page with no third-party code loaded at runtime. During development and in continuous integration the whole stack comes up under Docker Compose against a local emulator of the cloud services, and the identical code targets either the emulator or the live account depending only on which endpoint and credentials are configured.

A. Automated Test Suite

The tests run on the Node.js runtime's own test runner, with no third-party framework. Because every piece of cloud-facing code takes its client as a parameter, the unit tests feed it small handwritten fakes instead of a real service or the emulator, which keeps the suite quick, repeatable, and independent of the network. Across the four modules the suite comes to 115 tests, split as Table II shows. The fog module holds the most of them, since it is where the windowing, the statistics, and the compliance rules live, the code that most needs to be pinned down directly.

B. Continuous Integration

Every push and every pull request triggers a continuous-integration workflow of two jobs, one depending on the other. The first job installs dependencies and runs the unit suites of all four Node.js modules. The second runs only if the first succeeds: it stands up the emulator-backed Docker Compose stack, runs an end-to-end check that drives data through the sensors, the gateway, the queue, the ingestion function, and the table, and then tears everything down. Making the integration job wait on the unit job keeps the usual feedback quick, since a logic slip is caught by the fast unit suite, yet still puts the assembled system through its paces before any change is accepted.

C. Infrastructure-as-Code Deployment

Deployment was to a real provider laboratory account in the North Virginia region. All provisioning is infrastructure as code, applied inside an isolated workspace so the plan considers only this project's resources: one apply brought up twenty-four resources with no hand-run commands, the plan showing twenty-four to add and nothing to change or destroy. What that apply created is a key-value table, a message queue, the ingestion and dashboard functions fronted by a REST interface gateway, an edge compute instance whose firewall opens only the one gateway port and which holds no login key pair, a fixed public address bound to that instance, and two object-storage buckets, one hosting the dashboard frontend and one for staging the deployment.

D. A Deployment-Only Defect

One defect did not appear in any unit test or in local Docker Compose, and surfaced only once the dashboard function was invoked on the serverless runtime. To keep the dashboard code testable, its request handler accepts an optional third argument carrying pre-constructed cloud clients, which the unit tests supply as fakes; in production the handler is meant to build its own clients instead. The serverless runtime, however, invokes a handler with three arguments of its own, an event, a context, and a completion callback. The handler had treated any truthy third argument as injected clients, so on the real runtime it received the runtime's callback in that position, tried to read a client off it, and failed with an undefined-property error on the first request, even though every local test passed. The fix narrows the condition: the third argument is treated as injected clients only when it actually carries a store client, and otherwise the handler builds its own, so the fake-client path the tests rely on and the self-constructing path production needs are distinguished by the shape of the argument rather than merely its presence. The episode is a reminder that a runtime's own calling convention is part of a function's contract, and that behaviour visible only in the deployed environment must be verified there, not inferred from local tests alone.

The application code and the infrastructure-as-code configuration are documented in the repository at [GitHub repository URL]. The system was then evaluated by measuring the

running deployment, not by inspecting code alone; all figures below were read from the live endpoint.

IV. Live Evaluation

A. End-to-End Liveness

Roughly a minute after the edge instance finished bringing up its containers, the health endpoint showed all four indicators true, the gateway reachable, the queue reachable, the ingestion function active, and the pipeline current, with the newest stored window only a few seconds old. When that age sits in single digits, a reading taken at a plant has already passed through the sensor, the gateway, the queue, the ingestion function, and the table and is being served back on the dashboard, all inside a few seconds. It is exactly this that separates a genuinely connected pipeline from a collection of components that merely happen to be deployed.

B. Throughput and Accumulation

Polling the stored record count repeatedly through the check showed it rising poll after poll, which confirms the ingestion function was draining the queue and writing on an ongoing basis rather than just once. The count query itself walks the table's continuation cursor and adds up the per-page counts instead of issuing one count-only scan, because a lone scan returns only the first page and would quietly undercount once the table outgrew it; paging keeps the figure accurate at any table size. Across the same interval the plants endpoint served live data for both plants, five current readings each with their window minimum, maximum, and mean, and every plant carried a compliance badge derived from the edge rules. All static frontend assets were fetched successfully from object storage, the cross-origin header was present on the interface responses, and the page's inline interface origin had been correctly replaced with the real gateway address when the assets were uploaded.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows the plant-readings table with both plants' live values and range meters, the two per-plant compliance badges, the panel of window-average trend charts, and the header indicators reporting the pipeline healthy. The screenshot shows that the system is not just reachable from a command-line probe but serves correct, live, compliance-annotated content to an operator.

C. Cost

Standing cost stays low by design. Apart from the edge instance, every service in the pipeline bills per request and, at the volumes seen here, remains inside the provider's free tier, the queue, the two functions, the key-value store, the interface gateway, and the object storage all charging only on use. The edge instance is the sole always-billed piece, and since the serverless dashboard and interface do not need it running, only fresh readings and the gateway's own health check do, it can be halted while idle without pulling the dashboard down. Idle, then, the system costs essentially one small instance, and even that can be paused.

D. Limitations

Three limitations are worth stating outright. First, the sensor and fog tier sits on one edge instance and is therefore a single point of failure for that tier: stop the instance and new readings stop, although the already-stored data and the serverless read path stay up. A production build would spread the fog tier over several instances or availability zones. Second, because the laboratory account is session based, only single-session behaviour was checked; longer-run stability and credential rotation across many hours were not put to the test. Third, as the security discussion noted, the two functions share one broadly scoped account role; the intended split into least-privilege roles could not be done in an account barred from creating roles.

V. Conclusion

The aim of this work was to shift water treatment monitoring away from occasional sampling toward continuous, compliance-aware observation, using a distributed design that puts each task where it fits: windowing, summarisation, and rule checks on a fog tier at the plant, and durable storage with elastic serving in the cloud. What emerged is a working pipeline covering two plants and five streams apiece, raising turbidity, chlorination, acidity, and pressure breaches at the edge in the cycle they occur and showing a live, compliance-annotated operator view served from a real cloud deployment.

A few lessons stand out. The message queue earned its place not through latency but through resilience: separating a shaky plant link from the cloud consumer is what lets the pipeline ride out the conditions monitoring has to endure. The sort key was a lesson in how much a small modelling choice matters, since attaching the plant label to a timestamp is all that stands between two plants coexisting and one quietly overwriting the other. The sharpest lesson came from the deployment-only defect, where a handler that cleared every local test still fell over on its first real call because the serverless runtime's own calling convention clashed with a test seam, proof that only live verification against the real account, not local testing on its own, establishes that a serverless system works. Sensible next steps are to replicate the fog tier for availability, break the shared role into least-privilege roles, and move the edge rules from fixed thresholds toward trend-based detection, so a slow chlorine decline or a gradual turbidity rise is caught from its trajectory rather than only when it crosses a line.

References

[1] World Health Organization, *Guidelines for Drinking-water Quality, 4th ed. incorporating the 1st addendum*. Geneva: World Health Organization, 2017.

[2] N. Vijayakumar and R. Ramya, "The real time monitoring of water quality in IoT environment," in *Proc. Int. Conf. Innovations in Information, Embedded and Communication Systems (ICIIECS)*, 2015.

[3] S. Geetha and S. Gouthami, "Internet of things enabled real time water quality monitoring system," *Smart Water*, vol. 3, no. 1, 2018.

[4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

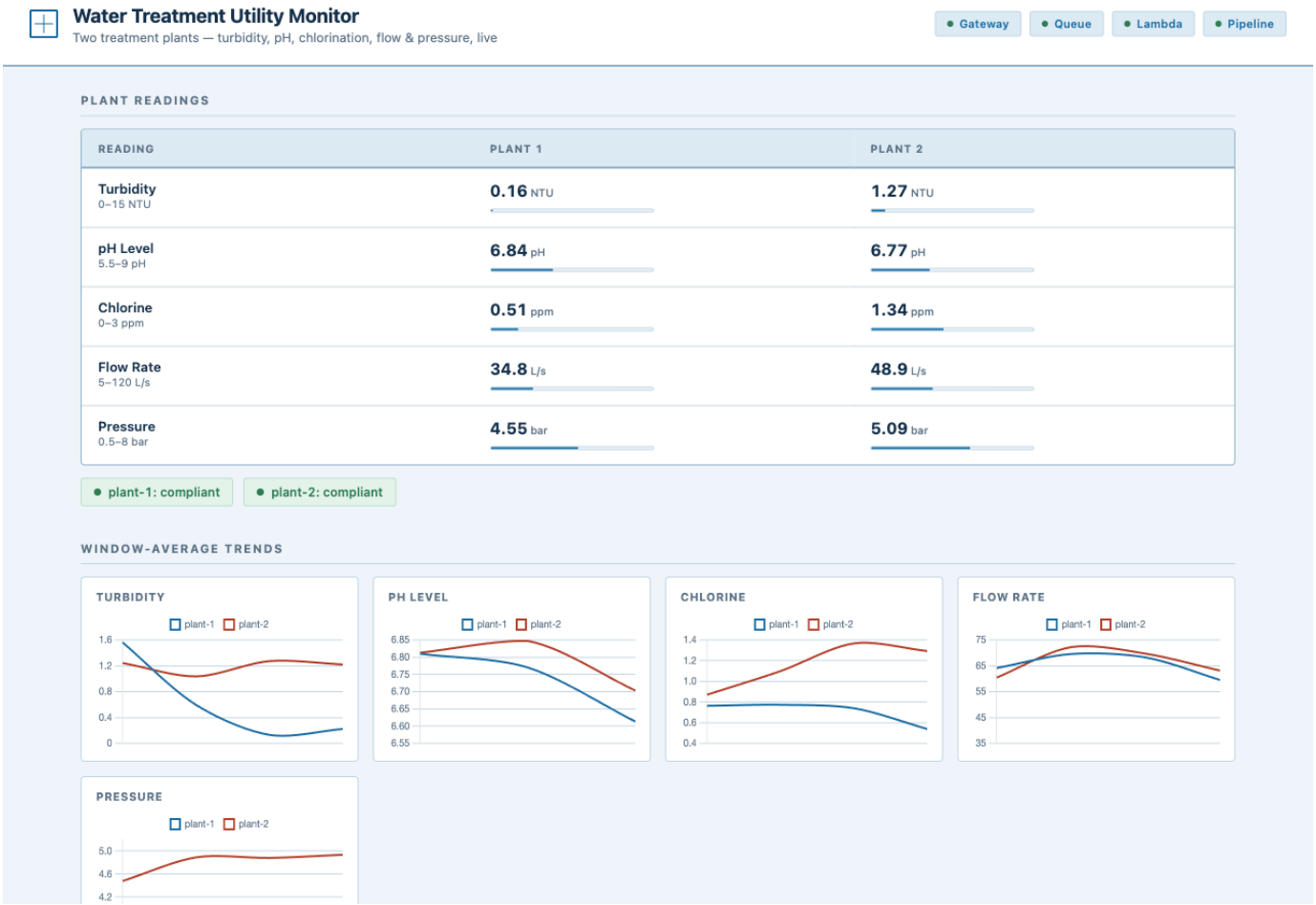

Fig. 2. The deployed operator dashboard rendered live in a browser. The upper table shows each stream's live value and a meter against its plausible range for both plants, with a per-plant compliance badge; the lower panel compares window-average trends across the two plants; the header carries the four pipeline health indicators.

[5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.

[6] R. K. Naha, S. Garg, D. Georgakopoulos, P. P. Jayaraman, L. Gao, Y. Xiang, and R. Ranjan, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47 980–48 009, 2018.

[7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.

[8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

[9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.

[10] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sosothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.

[11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.

[12] A. Barth, "The web origin concept," Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.